Cache write hit policies

Sparsh Mittal

Why a write-hit policy is required

A cache holds the copy of a block from main memory

A read-operation is not a problem, because it does not change the value of data-block

But, on a write-operation, the value of a block changes. If we don't do something, the value in the cache and memory would become inconsistent.

1. Write-Through

On a write-hit in this cache, also update the lower-level cache

Advantage: (1) This cache always remains consistent with its lower-level cache. (2) When a block is replaced, we need not update lower-level cache.

Disadvantage: It increases effective latency of write operations
Example: Assume L1 cache use write-through and there is no L2 cache. Base CPI = 1 and 10% of instructions are stores, write to memory takes 100 cycles.

Effective CPI = $1 + 0.1 \times 100 = 11$

2. Write-Back

On data-write hit, just update the block in this cache. And set a flag (called dirty bit). Do not update this block in lower-level cache.

- The dirty bit keeps track of whether the data of block is different from that in the lower-level cache.

When a dirty block is replaced

- Write it back to memory

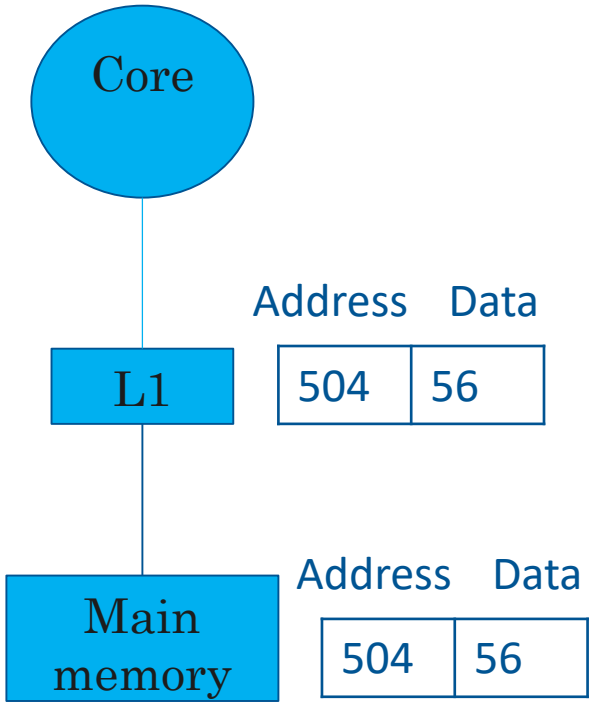
Summary of Write Policies

Write-through

- Update both this-cache and lower levels at the same time
- Lower levels always have most-recent value
- Simplifies replacement, but leads to large latency

Write-back

- Update only this-cache
- Update lower level caches when block is replaced
- Need to keep more state (dirty bit).
- The value stored in lower level may not be correct



Core writes 98 at address 504

504	98

504	56

Problem: inconsistent data

Solution 1: While writing L1 cache, also write main memory

504	98

504	98

Write-through cache

Solution 2

Write in L1 cache and set dirty flag. Do not update main memory

504	98	1

Dirty

504	56

When an address is replaced from L1 cache, check dirty flag. If this flag is set, then update main memory

378	901	0

Dirty

(A new block comes in L1 cache)

504	98

Write-back cache

For simplicity, we assume there is no L2 cache.

Changed block is shown by red color

Write-through L1 cache

L1 CACHE

		Tag	Data						
Initial state									
Write 34 at P	Miss	P	34						
Read Q	Miss	P	34	Q	12				
Read S	Miss	P	34	Q	12	S	56		
Write 99 at T	Miss	P	34	Q	12	S	56	T	99
Write 38 at Q	HIT	P	34	Q	38	S	56	T	99
Write 81 at Q	HIT	P	34	Q	81	S	56	T	99
Write 5 at Q	HIT	P	34	Q	5	S	56	T	99
Read U	Miss	U	45	Q	5	S	56	T	99
Read V	Miss	U	45	Q	5	V	83	T	99
Write 50 at P	Miss	U	45	Q	5	V	83	P	50
Read S	Miss	U	45	S	56	V	83	P	50

MAIN MEMORY

Addr	Data										
P	21	Q	12	S	56	T	19	U	45	V	83
P	34	Q	12	S	56	T	19	U	45	V	83
P	34	Q	12	S	56	T	19	U	45	V	83
P	34	Q	12	S	56	T	19	U	45	V	83
P	34	Q	12	S	56	T	99	U	45	V	83
P	34	Q	38	S	56	T	99	U	45	V	83
P	34	Q	81	S	56	T	99	U	45	V	83
P	34	Q	5	S	56	T	99	U	45	V	83
P	34	Q	5	S	56	T	99	U	45	V	83
P	34	Q	5	S	56	T	99	U	45	V	83
P	50	Q	5	S	56	T	99	U	45	V	83
P	50	Q	5	S	56	T	99	U	45	V	83

The dirty block is shown by ◯

Write-back L1 cache

L1 CACHE

		Tag	Data						
Initial state									
Write 34 at P	Miss	P	34						
Read Q	Miss	P	34	Q	12				
Read S	Miss	P	34	Q	12	S	56		
Write 99 at T	Miss	P	34	Q	12	S	56	T	99
Write 38 at Q	HIT	P	34	Q	38	S	56	T	99
Write 81 at Q	HIT	P	34	Q	81	S	56	T	99
Write 5 at Q	HIT	P	34	Q	5	S	56	T	99
Read U	Miss	U	45	Q	5	S	56	T	99
Read V	Miss	U	45	Q	5	V	83	T	99
Write 50 at P	Miss	U	45	Q	5	V	83	P	50
Read S	Miss	U	45	S	56	V	83	P	50

MAIN MEMORY

Addr	Data										
P	21	Q	12	S	56	T	19	U	45	V	83
P	34	Q	12	S	56	T	19	U	45	V	83
P	34	Q	12	S	56	T	19	U	45	V	83
P	34	Q	12	S	56	T	19	U	45	V	83
P	34	Q	12	S	56	T	99	U	45	V	83
P	34	Q	12	S	56	T	99	U	45	V	83
P	34	Q	12	S	56	T	99	U	45	V	83
P	34	Q	12	S	56	T	99	U	45	V	83
P	34	Q	12	S	56	T	99	U	45	V	83
P	34	Q	12	S	56	T	99	U	45	V	83
P	34	Q	12	S	56	T	99	U	45	V	83
P	50	Q	12	S	56	T	99	U	45	V	83
P	50	Q	5	S	56	T	99	U	45	V	83

Use of dirty-bit allows absorbing all writes to Q in L1 cache, so that only one write is required in main memory for 3 writes to Q in L1 cache.

Solved question

We have a cache with one set and 4 ways. We start with a cache that is already filled: having blocks P, Q, S, and T.

Now, the following accesses come:

Write at S. Write at S. Write at P. Write at S. Write at S. Read from T. Read from U. Read from V. Write at W. Write at Z

For above 10 accesses, how many write-operations will happen to memory for

(i) write-through cache (ii) write-back cache (no block is initially dirty).

Solved question: Write-through cache

First five writes will lead to memory-writes.

Then, read from T does not lead to any memory-write.

Next four operations lead to misses. One-by-one, they replace, Q, P, S and T from cache. They need not be written-back to memory, because it is a write-through cache. But, because the operations are write at W and write at Z, so we need to write to memory.

So, answer is 5+2 = 7 writes.

Solved question: Write-back cache

First five writes are absorbed in cache and corresponding dirty-flag is set. No write to memory

Then, read from T does not lead to any memory-write.

Next four operations lead to misses. One-by-one, they replace, Q, P, S and T from cache. Of these, only S and P need to be written back to memory.

Because the operations are write at W and write at Z, so we need to write to memory for them also.

Total = 2+2 = 4 writes.